# Aggregation Oriented Databases




pm jat @ daiict




# Relational "Reign"

- Relational databases, aka "SQL Databases"
  - The term "SQL Databases" came in use after non-relational (so called NoSQL has emerged)
- Relational almost ruled the industry for about four decades
  - Even though many things changed: programming languages, architectures, platforms, and processes;
  - it remained till the time "No SQL" appeared!
- What has been great about Relational databases that they ruled for decades?
- Why?

# Relational “Reign”

- Below are major reasons that Relational Systems ruled the industry for several decades:
- “**Simple (mathematical) data model**” providing transparency to File Systems and “Data File”
- **SQL**! Yes SQL is yet another reasons RDBMS kept remaining a choice of developers
- **Robust Relational Database Management Systems**
  - Integrated, **Consistency**, **Query Optimization**, Transactional Support (**ACID**), etc.
- Programming Standardization like ODBC/JDBC/ORM




# Why Relational data model is no more "universal" solution?

Five Reasons!

1. "**Scale**": Relational Do not scale "horizontally" and "elastically" (Volume Challenge)
2. "Normalization" and "Partitioning" do not go well. Query Performances are seriously affected
3. "**Strict Schema**" is becoming a problem in "Big Data" (Variety Challenge)
4. Programming discomforts – "**Impedance mismatch**"
   - Data Objects structurally do not match with relational tables, and this requires lots of "boilerplate codes"
   - ORM tools (like hibernate) do help quite a extent but brings in "complexity" and performance issues!
5. "**Data loading**" is slow – load time validation of schema and other constraints makes loading slow (Velocity Challenge)




# Let us talk about – one by one

- **Strict Schema**: benefits and problems that they bring in
- "**Impedance Mismatch**" as a problem!
- "**Aggregation Oriented Databases**"




# Strict Schema

- What is Schema and What is Strict Schema
  - Structured, Un-Structured, Semi-Structured Data?
  - Benefits and Problems of Schema
  - Flexible Schema
  - “Implicit” Schema
- Schema Enforcement/Validation
  - Load Time
  - Query Execution Time




# What is Schema in databases?

- Classically, often defined as
  - “Data Structure” + “Database Constraints”
- However, more generally, we can describe it as following
  - How database entities are to be described in the database, that is data items (attributes) and their “Existential Rules”
  - Value Validity Rules for Data Items (Domain for an attributes),
  - Data Item being optional or mandatory
  - Value of Data item being Unique or Non-Unique
  - Relationship between entities
  - Relationship Cardinality and Participation being Mandatory or Optional, etc.

# What is “Fixed Schema” or “Strict Schema”

- Schema is to be defined upfront, that is before any data is added to it.
- DBMS ensures that any data added to a database comply the structure and other constraints as per the schema
- Data Validity of data is done before putting into database; that is why, there is also term “Load time schema validation”. Validation here mean validation of data with its schema!
- Any data that violates “schema” are denied to be added into the database!
- **Advantage of “Fixed Schema”**
  - Data are highly consistent
  - Helps in better planning of “data storage” and ultimately helping in efficient execution of queries!




# Problems that Fixed Schema bring in?

- In traditional transaction processing environment, “fixed schema” has been at the advantage, however
- “Fixed Schema” is becoming critical in modern computing
  - **Figuring out schema in advance is becoming impractical**
    - More and more column may get added with the time
    - Some columns may be becoming irrelevant with the time
  - Fixed schema leads to “too many nulls” for an entity. (All entities together may have too many attributes where as individual entities have values for few attributes). **Makes the sparse data**!
- Data Loading in fixed schema becoming undesirably slow because of “**Load Time Validations**”!




# Schema Validation

- What it is?
- Why it has been important?
- What are the problems associated with it?
- “Load Time Schema Validation” and “Run-time schema Validation”




# What if we do not specify Schema at all

- Everything is not good in “Schema-lessness”
  - “invalid data – types, range etc.
  - “Invalid References (FK)”
- More Run-time errors because of invalid data
- **Totally schema-less can be disastrous**?
  - Two data objects having two names quantity attribute: “qty” for “quantity”; how do we reconcile this?
  - “3” and “three” as value for an attributes
- After appearances of NoSQL databases, we have concept of “**Partial Schema**” or “**Flexible Schema**”, even “**No Schema**”!




# “Flexible Schema”

- In “Schema less” or “Flexible Schema” database systems, following is what practiced

  **No schema defined upfront!**

- When data to be **processed has no schema attached to it**
- We have already seen systems like Spark-SQL where
- Schema is not defined Upfront and no schema validation is done at “data load” time that is when data is put in database for performing operations.
- In such cases, application defines some schema and expect data to comply that schema. The application checks correctness of data for their validity against the application defined schema




# “Flexible Schema”

- Flexibility in “Flexible schema” can come from:
- **(1) Partial Schema Definition**: few attributes are specified, instead of defining for all attributes. But actually data records may have values for additional attributes
  - Schema may not be specified at all: for example in Dynamo-DB only Key is specified, other than this no info!
- **(2) Schema Validation may not be enforced at “load time”**
  - That is ignore constraints while adding/updating data records
  - Schema enforcement can be only for adding new data but not for existing data!

# Notion of “Implicit Schema”

- When programs use “schema less” entities or with “flexible schema”, how does data record is “parsed and read as data object”?
- Application logic makes certain assumptions about the schema, and programs accordingly. If any incompatibility found in the code, program would crash at run-time
- Let this “Assumptive Schema” be called as “Implicit Schema”
- Such a scheme is often referred as “implicit schema” (expected schema of data being read)




# Let us talk about – one by one

- **Strict Schema**: benefits and problems that they bring in

==> "**Impedance Mismatch**" as a problem!

- "**Aggregation Oriented Databases**"

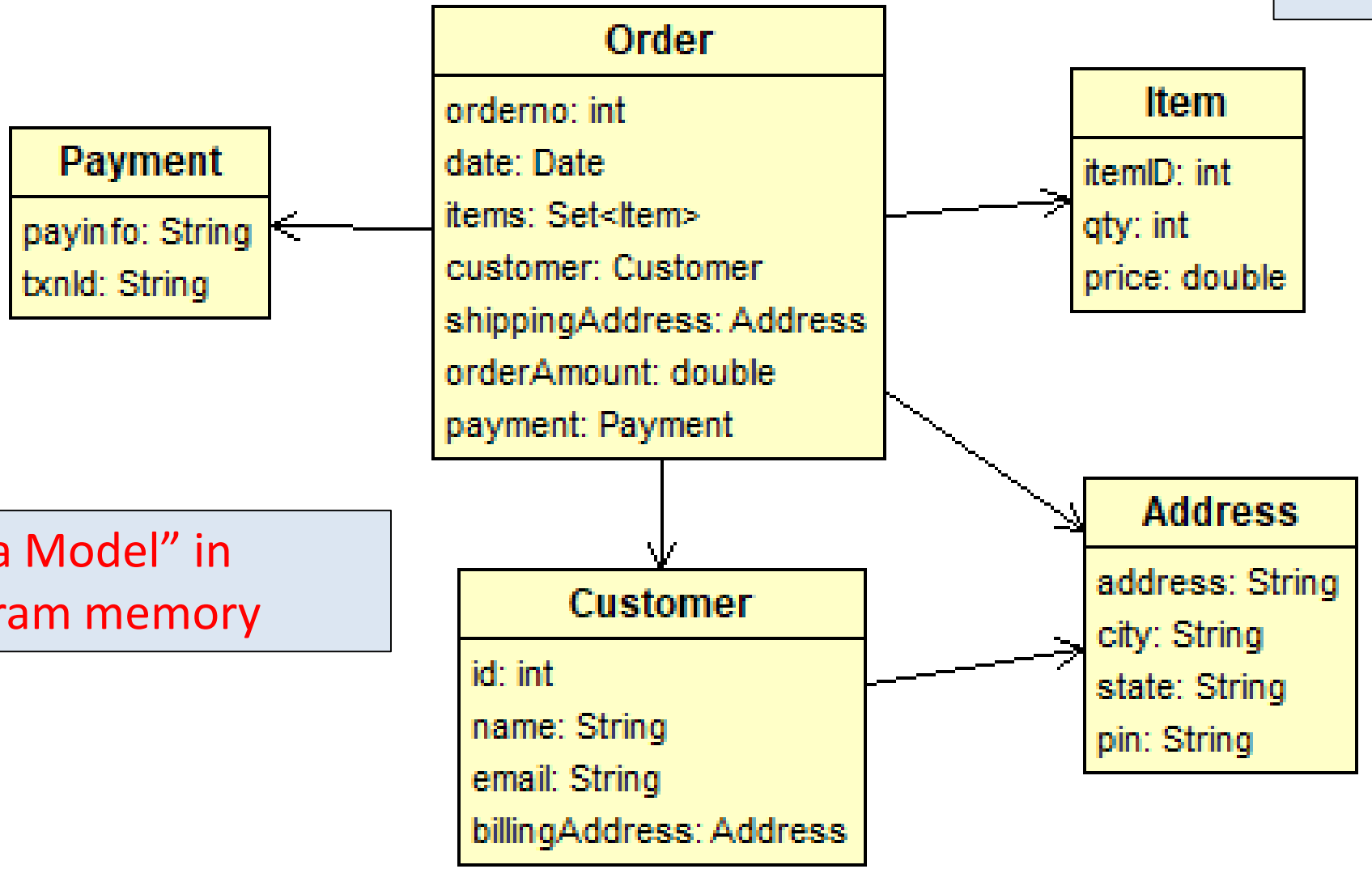



# "Impedance Mismatch" as a problem!

- Consider a scenario of "Order Management of **My Amazon**"
- Let us say we have following data objects with specified data fields that are to be saved in database.
  - Customer: ID, Name, email, Billing Address
    - Customer places orders
    - Customer has multiple Addresses
  - Order: Order No, Date, Order Amount, customer
  - Order has a list or Item objects, and each Order Item holds data: Item-ID, Item-Name, Item Category, Qty, Price, so forth

# An Ordering System

Hope this diagram makes sense to you?




"Data Model" in program memory



```java
public class Payment {

    private String payinfo;
    private String txnId;
```

```java
public class Address {

    private String id;
    private String address;
    private String city;
    private String state;
    private String pin;
}
```

```java
public class Item {

    private int itemID;
    private int item_name;
    private int category;
    private int qty;
    private double price;

}
```

```java
public class Customer {

    //business operations to be added

    private int id;
    private String name;
    private String email;
    private Address billingAddress;
```

```java
public class Order {

    private int orderno;
    private Date date;
    private Set<Item> items;
    private Customer customer;
    private Address shippingAddress;
    private double orderAmount;
    private Payment payment;
```

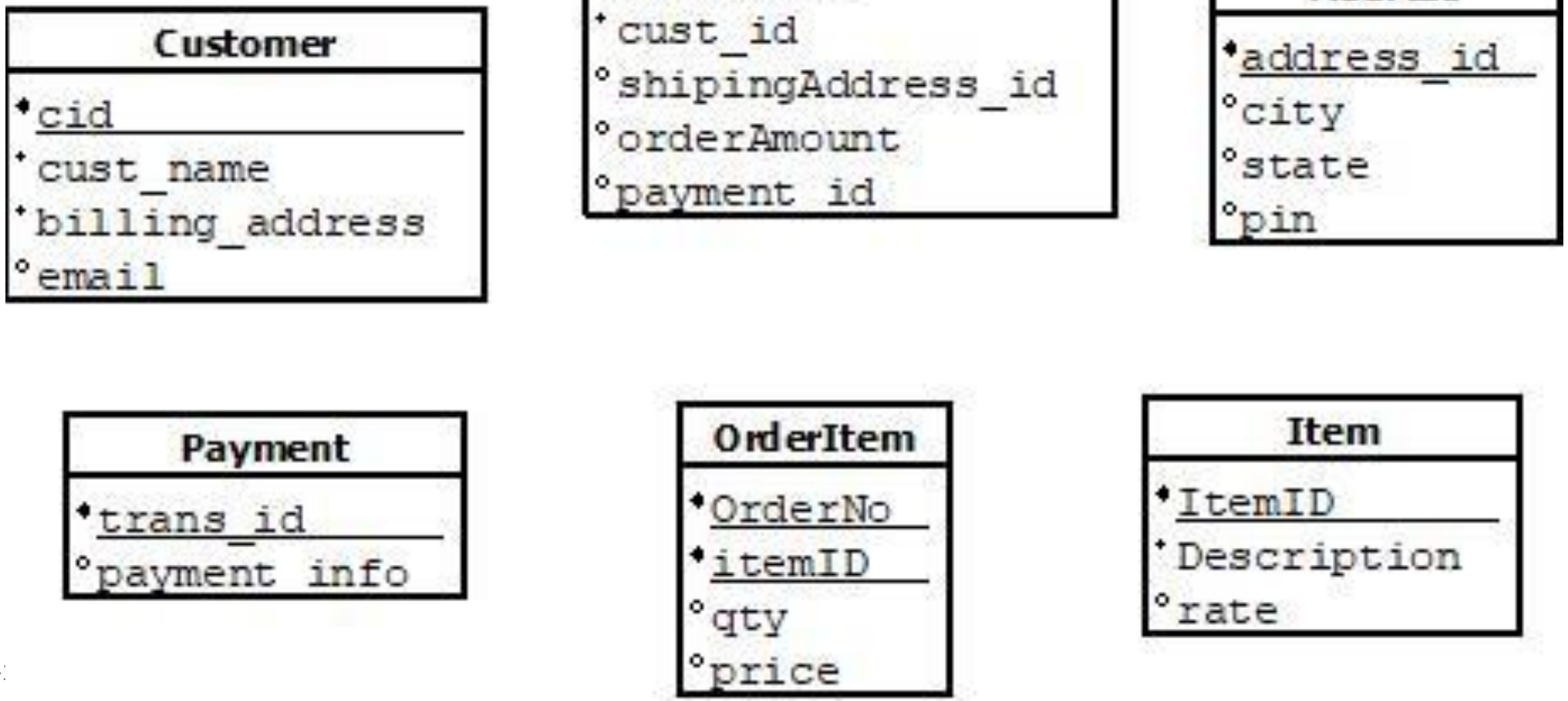


A sample Order object in JSON representation

```
{
  "order": {
    "orderNo":123,
    "customer": {
      "id": 1,
      "name": "Mayank",
      "email": "mayank@gmail.com",
      "billingAddress": {"city": "Bangalore"}
    },
    "orderItems":[
      {
        "productId":27,
        "price": 325,
        "cat":"book",
        "productName": "NoSQL Distilled"
      },
      {
        "productId":19,
        "price": 550,
        "cat":"computer accessories",
        "productName": "Pen Drive : 128 GB"
      }
    ],
    "shippingAddress":[{"city":"Ahmedabad"}],
    "payment": {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft"
```

Problem Context:
How do we store this in database?

# Relational Tables for " … My Amazon"





# Saving Object in relational Table is real pain !!!

```
void save_order(Order order) {

    if !customer_exists() {
        //INSERT INTO Customer Table by collecting data from order object
        cust_id = order.getCustomer().getId()
        cust_name = order.getCustomer().getName
        //so on

        //build a dynamic "INSERT INTO CUSTOMER ..." SQL statement and execute
    }

    //Save Order Row
    //build a dynamic "INSERT INTO ORDER ..." SQL statement and execute
    //so forth
    //should throw exception if already there!

    //Add rows to Order Items
    //build a dynamic "INSERT INTO ORDERITEMS ..." SQL statement and execute
    //so forth

}
```

INSERT INTO CUSTOMERs if not there?

INSERT INTO ORDER

INSERT INTO ORDER - ITEMS

UPDATE PRODUCTS for depleted STOCKS



# Reading Object from Tables is hard !!!

```
Order read_order(int order_no) {

    Order order = new Order();

    //Read row from Order table and populate order object
    sql = "select * from order where order_no=" + order_no
    result = execute(sql);
    order.setId( order_no );
    order.setDate( result.getColumn("date") );
    order.setAmount( result.getColumn("order_amount") );
    customer_id = result.getColumn("customer_id")

    //Read related cutomer row, construct and populate a cust
    Customer customer = new Customer()
    customer.setID( customer_id )
    customer.setName( queryresult.getColumn("name") )
    customer.setEmail( queryresult.getColumn("email") )
    //
    order.setCustomer( customer );
    //so on

    return order;
```




# Impedance Mismatch!

- Problems are mainly due to structure mismatch between data object in memory and its storage in tables.
  - "Aggregated Objects" and "Normalized Tables"
- It is referred as "impedance mismatch"
- Does it make it clear that "what is impedance mismatch"?
- Though JPA tools like hibernate makes life much easy, still problem remains!




# Impedance Mismatch

- People hoped that some thing like Object Oriented Databases would be replacing relational databases, but it did not happen
  - Probability OODB could not become success due to its own complexities, and could not provide many of RDB features
- Though most translations (Object to Relations and Relations to Objects) are automated by Object Relational Mapping (ORM) tools like Hibernate or so; they bring in their own complexity, maintenance and computational cost.
- A nice critic of ORM is available from Martin Flower at https://martinfowler.com/bliki/OrmHate.html
- No SQL Databases help here!




# Let us talk about – one by one

- **Strict Schema**: benefits and problems that they bring in
- "**Impedance Mismatch**" as a problem!

==> "**Aggregation Oriented Databases**"




# Aggregation Oriented Databases

- What is “Aggregation” here?
  - Understand “aggregated object” or simply “aggregate”
- Define “Aggregation Oriented Databases”
- What are their Benefits of Aggregation Oriented Databases?
- What are drawbacks of Aggregation Oriented Databases?
- Intuition of “Designing Aggregation Oriented Databases”!



# Example "**Aggregated Object**"

```
{
  "order": {
    "orderNo":123,
    "customer": {
      "id": 1,
      "name": "Mayank",
      "email": "mayank@gmail.com",
      "billingAddress": {"city": "Bangalore"}
    },
    "orderItems":[
      {
        "productId":27,
        "price": 325,
        "cat":"book",
        "productName": "NoSQL Distilled"
      },
      {
        "productId":19,
        "price": 550,
        "cat":"computer accessories",
        "productName": "Pen Drive : 128 GB"
      }
    ],
    "shippingAddress":[{"city":"Ahmedabad"}],
    "payment": {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft"
```

- Represents the "Order" in JSON format, and
- Embeds:
  - Customer object
  - List of Order-Items
  - Address object
  - Payment




# Concept of "Aggregated Object"

- Normally, when programming in language like OOP, objects are house related object within
  - "Aggregation" in OOP word!
- This basically mean, related objects are stored within the object!
- In entity terms, we say that an entity stores related entities within
- Order object shown on next slide stores
  - Customer Object, Order Items, etc.
- As a result we have only single data object to store all the data!
- Let us call this an "**Aggregated Object**" or simply "**Aggregate**"!




# Options for storing “Aggregated Object”

- In program memory, objects often, have been stored as “aggregated objects”!
- However, come to the database, we store in relational database, and
  - **Objects are flattened into relational tab;e rows and then INSERTED INTO respective tables**
- Having realized that we need to look for alternate options for the said reasons,
- Other approach, we have is **store aggregate as such as a single object** in the database.
- In this a case, database becomes “**Collection of Aggregates**” and hence
- Called as **Aggregation Oriented Databases**!

# “Aggregation Oriented Databases”

- **Database that allow storing “aggregated objects” is called “Aggregation Oriented Database”.**




# Benefits of "Aggregation Oriented Databases"

- "**Partition Friendly**": since database is now "Key" oriented, that is "aggregate key", they can be partitioned and distributed based on "aggregate key"
- Since all related data are also stored together (as aggregate) => "**Localized Computation**" => faster execution of operations
  - We do not require joining for fetching related data. Why?
  - The number of operations gets reduced: For example, adding a new "Order" would require just one INSERT operation (rather than applying INSERT operation to multiple tables).
  - Same for Update/Remove
- Addresses the problem of "impedance mismatch"




# “Aggregation” and “Normalization”

- Relational Database stores “Normalized tables”
- In a sense, aggregation can be referred as reverse of “Normalization”
- “Aggregation” “embeds” related entities within, whereas normalized design keeps the related entities separated, therefore,
- “Aggregated data” design can also be referred as “De-Normalized” design!
- Another terms, popular in no-sql systems is “embedding” and “referencing”
  - Embedding refers to “Aggregation”, where as
  - “Referencing” refers to “Normalization”

# Downside of “Aggregation Oriented Databases”

- Since data objects here are not normalized there would be possibility of “redundancy”
- “Querying” sometimes becomes complex, particularly when looking for “subpart” of an aggregate
- For example, querying orders for a given date range or for a customer?
- **Storing Aggregates as such in a database suffers from being efficient for some set of queries while inefficient for another set of query**
  - You do not have this problem in relational systems. Why?




# Design Challenge in "Aggregation Oriented Databases"

- For the "Online Order" application, in the discussion here, Let us say we have the following two design options:
- Option #1 (Customer embeds related Order entities)
  - DB is a collection of "Customer Aggregates", and
  - Customer object* embeds all its order objects, and
  - Every Order embeds all related Order Item entities
- Option #2 (Order embeds related Customer entity)
  - DB is a collection of "Order Aggregates", and
  - Order object embeds related Customer object
  - Every Order embeds all related Order Item entities
- Which one should be chosen?

*You can read the term "object" here as an entity as well!



# Customer Aggregate

- A database is a collection of Customer Objects (Aggregates)
- The customer object embeds all of its orders. Order object embeds all related object data including date, addresses, items, and payments.


```
{
  "customer": {
    "id": 1,
    "name": "Mayank",
    "email": "mayank@gmail.com",
    "billingAddress": [{"city": "Bangalore"}],
    "orders": [
      {
        "orderNo":123,
        "customerId":1,
        "orderItems":[
          {
            "productId":27,
            "price": 325,
            "cat":"book",
            "productName": "NoSQL Distilled"
          },
          {
            "productId":19,
            "price": 550,
            "cat":"computer accessories",
            "productName": "Pen Drive : 128 GB"
          }
        ],
        "shippingAddress":[{"city":"Ahmedabad"}],
        "payment": {
          "ccinfo":"1000-1000-1000-1000",
```



# Order Aggregate

- The database is collection of Order Object (Aggregates)
- Order object embeds all object data including customer, address, items, payments.

- Customer Details shall be repeated for every Order of a customer. Correct?

```
{
  "order": {
    "orderNo":123,
    "customer": {
      "id": 1,
      "name": "Mayank",
      "email": "mayank@gmail.com",
      "billingAddress": {"city": "Bangalore"}
    },
    "orderItems":[
      {
        "productId":27,
        "price": 325,
        "cat":"book",
        "productName": "NoSQL Distilled"
      },
      {
        "productId":19,
        "price": 550,
        "cat":"computer accessories",
        "productName": "Pen Drive : 128 GB"
      }
    ],
    "shippingAddress":[{"city":"Ahmedabad"}],
    "payment": {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft"
```




# "Query Load" influences "Aggregation Oriented" databases!

- Normally NoSQL databases distribute their data on multiple computing nodes based on "Primary Key" or on some other attribute!
- Let us say, customer objects (aggregates) are distributed and stored based on Customer ID
- Therefore, searching on Customer ID will be **efficient**, whereas
- Search on "Order Number", Date/Time", and "Item" shall be **inefficient**.
  - Answering such a query shall require scanning the full database collection on all nodes. Because we can not figure out on what partition a given order number is
- How do we solve this problem? Any guess?
- Relational Systems do not suffer from such a problem. Why?




# “Query Load” influences “Aggregation Oriented” databases!

- Relational Systems do not suffer from such a problem because they are “Normalized”!
- How does normalization help here?
- To address the said problem in the aggregated database also, we do some normalization, as following:
  - Customer and Order are two different aggregates and have reference into other one. Or have references in both!
- No-SQL system have a design challenge of finding a threshold, somewhere in between of “embedding” and “referencing”!

- **Databases contains two collections: Customers and Orders**
- Orders are taken out of customer
- Order object has reference to customer objects (like the foreign key in relational, but referential integrity check is not done.
- Now allows processing orders in time space also

Customer aggregate

```
{
  "customer": {
    "id": 1,
    "name": "Mayank",
    "email": "mayank@gmail.com",
    "billingAddress": {"city": "Bangalore"}
  }
}
```

Order Aggregate

```
{
  "order": {
    "orderNo":123,
    "customerId":1,
    "orderItems":[
      {
        "productId":27,
        "price": 325,
        "cat":"book",
        "productName": "NoSQL Distilled"
      },
      {
        "productId":19,
        "price": 550,
        "cat":"computer accessories",
        "productName": "Pen Drive : 128 GB"
      }
    ],
    "shippingAddress":[{"city":"Ahmedabad"}],
    "payment": {
      "ccinfo":"1000-1000-1000-1000",
      "txnId":"abelif879rft"
    }
  }
}
```

# Bi-Directional references

- **Databases contains two collections**
- Both collections have reference to each other. Called bidirectional references.
- Eases out and querying becomes efficient!

Order aggregate

```
{
  "order": {
    "orderNo":123,
    "customerId":1,
    "orderItems":[
      {
        "productId":27,
        "price": 325,
        "cat":"book",
        "productName": "NoSQL Distilled"
      },
      {
        "productId":19,
        "price": 550,
        :"computer accessories",
        uctName": "Pen Drive : 128 GB"

        Address":[{"city":"Ahmedabad"}],
        : {
        :"1000-1000-1000-1000",
        :"abelif879rft"
    }
}
```

Customer aggregate

```
{
  "customer": {
    "id": 1,
    "name": "Mayank",
    "email": "mayank@gmail.com",
    "billingAddress": {"city": "Bangalore"},
    "orders": [{ "orderno" : 123}, { "orderno" : 154} ]
  }
}
```




# Normalization in "Aggregation Oriented" databases!

- Note that when we do normalization in "Aggregation Oriented" databases
- We require splitting a "order object" into two "persistent entities"
- Higher aggregation granularity requires less splitting of objects while saving them into the databases BUT THEN, they are more specific to a set of queries!

  …. always a challenge in "Aggregation Oriented Databases"




# "Aggregation Oriented" Databases

## Wrap Up

- If a database system allows us saving "aggregated" objects as such, the we call them as "Aggregation Oriented" Database system.
- Most No SQL database systems are aggregation oriented except graph databases.
- Aggregate oriented databases are biased towards a "query load" (querying use cases)
- Having appropriate aggregate is key in "Aggregation Oriented" database designs!

# References and Further Readings

[1] Chapter 1 of book Sadalage, Pramod J., and Martin Fowler. *NoSQL distilled: a brief guide to the emerging world of polyglot persistence*. Pearson Education, 2013.